Chapter 7 Support Vector Machines

Austin R.J. Downey

Support Vector Machines

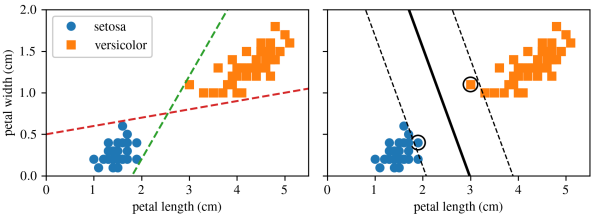
Support Vector Machines are geometric classifiers.

Key idea:

- Separate classes with a decision boundary
- Maximize the margin around that boundary
- Use nearby points to define the boundary

Those nearby points are called:

Support Vectors



Large margin classification.

Large Margin Classification

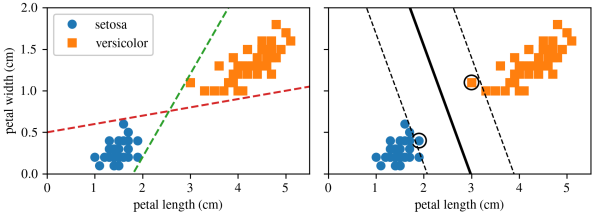
Many linear classifiers may separate the training data.

SVMs choose the classifier that creates the widest possible margin.

This margin is often called the:

street

Only points on the edge of the street directly affect the boundary.



The boundary is shaped by the support vectors.

SVMs Need Feature Scaling

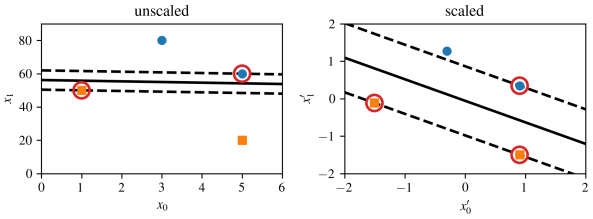
SVMs are sensitive to feature scale.

If one feature dominates the scale, the margin may be distorted.

A common preprocessing step is:

`StandardScaler`

Scaling usually produces a more appropriate decision boundary.



Effect of feature scaling.

Hard Margin Classification

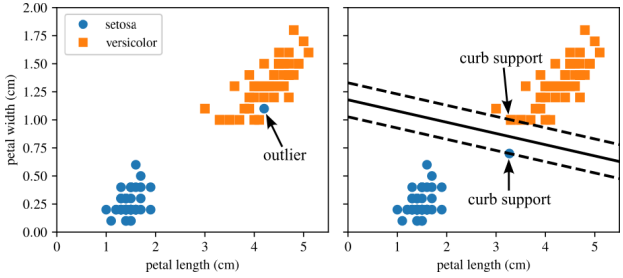
Hard margin classification requires:

- All points correctly classified
- No margin violations

Limitations:

- Only works for linearly separable data
- Very sensitive to outliers

A single outlier can make the hard-margin problem impossible or unstable.



Hard margin sensitivity to outliers.

Soft Margin Classification

Soft margin classification allows some margin violations.

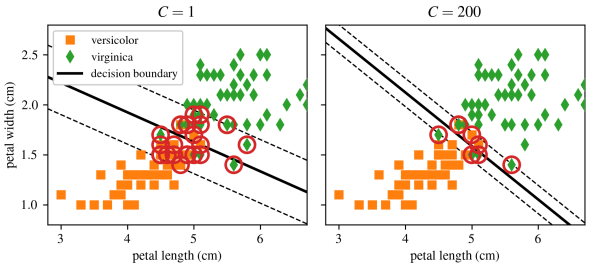
Goal:

- Keep the margin wide
- Limit classification errors
- Improve generalization

The balance is controlled by:

$$C$$

Smaller C gives stronger regularization.



Effect of the C hyperparameter.

The Role of C

The hyperparameter C controls the tradeoff between margin width and violations.

- Smaller C:
  - Wider margin
  - More violations allowed
  - More regularization
- Larger C:
  - Narrower margin
  - Fewer violations allowed
  - Less regularization

Note

Overfitting in an SVM can often be reduced by decreasing C.

SVM Notation

In earlier chapters, model parameters were often written as:

$$\boldsymbol{\theta}$$

For SVMs, we usually write:

$$\mathbf{w} \quad \text{and} \quad b$$

where:

- $\mathbf{w}$: weight vector
- b: bias term
- No extra bias feature $x_0 = 1$ is appended

The decision function is built directly from $\mathbf{w}$, $\mathbf{x}$, and b.

Decision Function and Predictions

A linear SVM computes:

$$\mathbf{w}^\top \mathbf{x} + b = w_1 x_1 + \cdots + w_n x_n + b$$

Prediction rule:

$$\hat{y} = \begin{cases} 0 & \text{if } \mathbf{w}^\top \mathbf{x} + b < 0 \\ 1 & \text{if } \mathbf{w}^\top \mathbf{x} + b \geq 0 \end{cases}$$

The decision boundary occurs when:

$$\mathbf{w}^\top \mathbf{x} + b = 0$$

SVM Decision Function

For two features, the decision function is a plane.

Important levels:

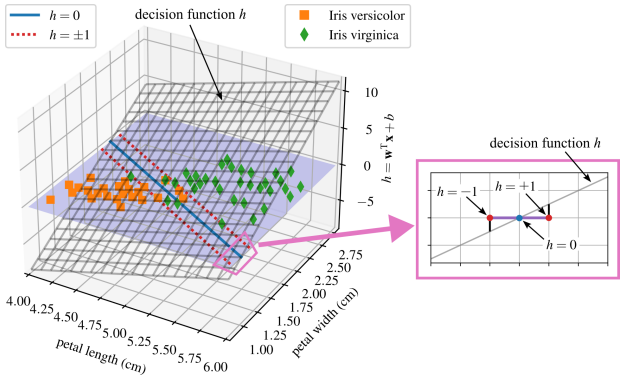
$$h = 0$$

defines the decision boundary.

$$h = 1 \quad \text{and} \quad h = -1$$

define the margin boundaries.

Training adjusts $\mathbf{w}$ and b to make the margin wide.



Decision function for the Iris dataset.

Training Objective

The slope of the decision function depends on:

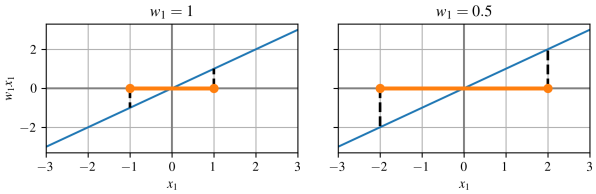
$$\|\mathbf{w}\|$$

Reducing $\|\mathbf{w}\|$ increases the margin.

Therefore, SVM training tries to:

$$\text{minimize } \|\mathbf{w}\|$$

This maximizes the margin between classes.



Smaller weights produce a larger margin.

Hard Margin Objective

Let $t^{(i)} = -1$ for negative samples and $t^{(i)} = 1$ for positive samples.

The hard margin constraint is:

$$t^{(i)} \left(\mathbf{w}^\top \mathbf{x}^{(i)} + b \right) \geq 1$$

The optimization problem is:

$$\begin{aligned} \underset{\mathbf{w}, b}{\text{minimize}} \quad & \frac{1}{2} \mathbf{w}^\top \mathbf{w} \\ \text{subject to} \quad & t^{(i)} \left(\mathbf{w}^\top \mathbf{x}^{(i)} + b \right) \geq 1 \end{aligned}$$

This enforces a margin with no violations.

Why Minimize $\frac{1}{2}\mathbf{w}^\top\mathbf{w}$?

The objective

$$\frac{1}{2}\mathbf{w}^\top\mathbf{w}$$

is equivalent to

$$\frac{1}{2}\|\mathbf{w}\|^2$$

Why use the squared norm?

- It is smooth and differentiable
- Its derivative is simple
- It makes optimization easier

Note

Minimizing $\|\mathbf{w}\|$ directly is harder because it is not differentiable at $\mathbf{w} = 0$.

Soft Margin Objective

Soft margin classification introduces slack variables:

$$\zeta^{(i)} \geq 0$$

Each $\zeta^{(i)}$ measures how much sample i violates the margin.

The optimization problem becomes:

$$\begin{aligned} \underset{\mathbf{w},b,\zeta}{\text{minimize}} \quad & \frac{1}{2}\mathbf{w}^\top \mathbf{w} + C\sum_{i=1}^{m}\zeta^{(i)} \\ \text{subject to} \quad & t^{(i)}\left(\mathbf{w}^\top \mathbf{x}^{(i)} + b\right) \geq 1 - \zeta^{(i)} \end{aligned}$$

The parameter C controls the penalty for violations.

Quadratic Programming

Both hard-margin and soft-margin SVMs are convex quadratic optimization problems.

General quadratic programming form:

$$\begin{aligned} \underset{\mathbf{p}}{\text{minimize}} \quad & \frac{1}{2}\mathbf{p}^\top \mathbf{H}\mathbf{p} + \mathbf{f}^\top \mathbf{p} \\ \text{subject to} \quad & \mathbf{A}\mathbf{p} \leq \mathbf{b} \end{aligned}$$

For a linear SVM:

- $\mathbf{p}$ contains the weights and bias
- The objective encodes margin maximization
- The constraints enforce correct classification or slack penalties

Example 7.1: Support Vector Machine Classification

- Train a linear SVM classifier on Iris petal features
- Scale the data before training
- Derive and plot the decision boundary
- Visualize the margins
- Identify margin violations and support-vector-like points

Performance is evaluated using:

- Confusion matrix
- F1 score

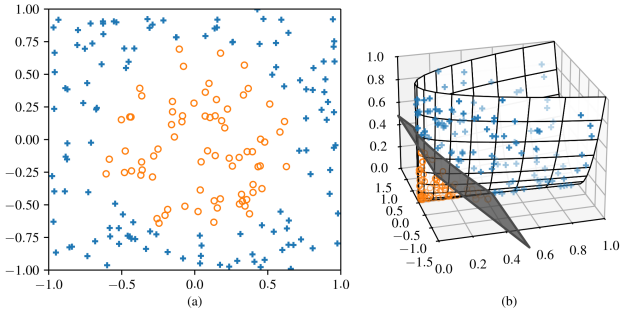
This example demonstrates large-margin classification on real data.

Nonlinear SVM Classification

Many datasets are not linearly separable in their original feature space.

A common strategy is to transform the data into a higher-dimensional feature space.

In the transformed space, a linear boundary may separate the classes.



Nonlinear data transformed to a separable space.

Adding Polynomial Features

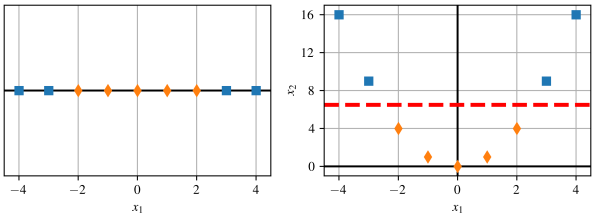
A simple feature transformation can make data linearly separable.

Example:

$$x_2 = x_1^2$$

The original one-dimensional data are not separable.

After adding x_2, the data become separable in two dimensions.



SVM in higher dimensions.

Polynomial Features with LinearSVC

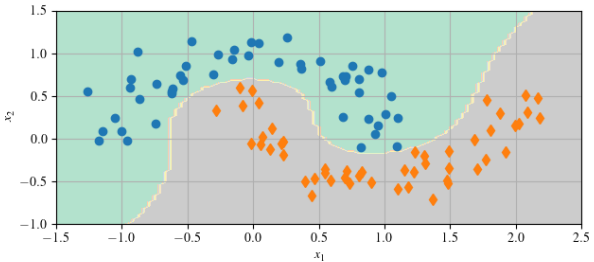
In Scikit-Learn, this can be implemented with a pipeline:

- `PolynomialFeatures`
- `StandardScaler`
- `LinearSVC`

This works well for datasets like:

`make_moons()`

The classifier is still linear in the transformed feature space.



SVM classifier with polynomial features.

Example 7.2: Nonlinear Classification

- Generate nonlinear data using `make_moons`
- Build a pipeline with:
  - `PolynomialFeatures`
  - `StandardScaler`
  - `LinearSVC`
- Train a linear SVM in the expanded feature space
- Visualize the resulting nonlinear decision boundary

This example shows how feature transformations can make nonlinear classification possible.

The Kernel Trick

Polynomial feature expansion can become expensive.

Problems:

- Low-degree features may underfit
- High-degree features may create too many variables
- Computation can become slow

The kernel trick avoids explicitly expanding the feature space.

It replaces:

$$\mathbf{x}^\top \mathbf{z}$$

with:

$$k(\mathbf{x}, \mathbf{z})$$

Kernel Trick Interpretation

The kernel function

$$k(\mathbf{x}, \mathbf{z})$$

computes similarity between two data points.

It allows the SVM to behave as if it were working in a higher-dimensional feature space.

Benefits:

- Curved decision boundaries
- No explicit feature expansion
- Powerful nonlinear classification

The kernel trick is implemented in:

`SVC`

Polynomial Kernel

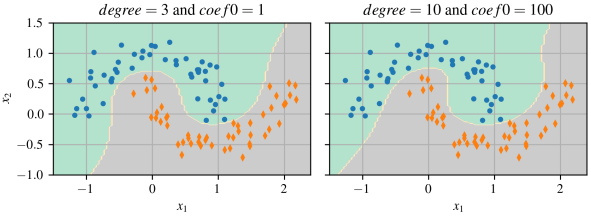
A polynomial kernel can produce nonlinear decision boundaries.

The degree controls model flexibility.

Higher degree:

- More flexible
- More risk of overfitting

The `coef0` parameter controls the influence of lower- vs. higher-degree terms.



Polynomial-kernel SVMs.

Tuning Kernel SVMs

Kernel SVMs can be sensitive to hyperparameters.

Common tuning approach:

- Start with a broad grid search
- Identify promising regions
- Refine with a narrower grid search

Important hyperparameters:

- C
- polynomial degree
- `coef0`
- γ for RBF kernels

Understanding each hyperparameter helps guide the search.

Standard Kernels

Three kernels cover most practical cases.

Polynomial kernel:

$$k_{\text{poly}}(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^\top \mathbf{z} + c)^d$$

RBF kernel:

$$k_{\text{rbf}}(\mathbf{x}, \mathbf{z}) = \exp\left(-\gamma \|\mathbf{x} - \mathbf{z}\|^2\right)$$

Sigmoid kernel:

$$k_{\text{sig}}(\mathbf{x}, \mathbf{z}) = \tanh\left(\kappa \mathbf{x}^\top \mathbf{z} + \theta\right)$$

Choosing a Kernel

A practical rule of thumb:

- Start with the RBF kernel
- Try polynomial kernels when interaction order matters
- Treat the sigmoid kernel as experimental

Note

Kernel methods require scaled features. Standardization is usually important before training.

The best kernel depends on:

- Dataset size
- Noise level
- Feature interactions
- Desired model flexibility

Example 7.3: Polynomial Kernel Trick

- Use `SVC` instead of `LinearSVC`
- Apply a third-degree polynomial kernel
- Classify nonlinear `make_moons` data
- Use a Scikit-Learn pipeline
- Visualize the curved decision boundary

This example shows how the kernel trick produces nonlinear classification without explicitly building polynomial features.

Computational Complexity

Scikit-Learn provides several SVM-related classifiers.

class	time complexity	out-of-core	kernel trick
`LinearSVC`	$O(mn)$	no	no
`SVC`	$O(m^2n)$ to $O(m^3n)$	no	yes
`SGDClassifier`	$O(mn)$	yes	no

Scaling is required for all three methods.

LinearSVC vs. SVC

`LinearSVC`

- Uses `liblinear`
- Linear decision boundary only
- Scales approximately as $O(mn)$
- Good for large, high-dimensional datasets

`SVC`

- Uses `libsvm`
- Supports kernels
- Can model nonlinear boundaries
- Slower for large datasets

Support Vector Regression

Support Vector Regression adapts SVMs to continuous outputs.

Instead of separating classes, SVR fits a prediction curve surrounded by a tube.

Tube width:

$$\epsilon$$

Points inside the tube do not affect the loss.

The regularization parameter C controls how strongly violations outside the tube are penalized.

Linear SVR

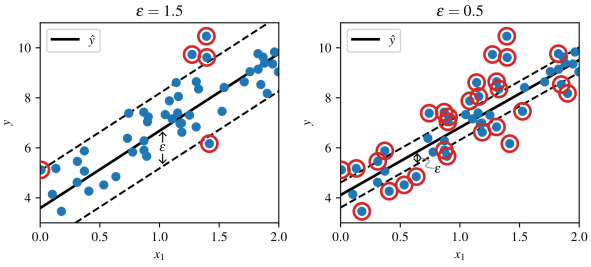
Linear SVR fits a linear function with an ϵ-insensitive margin.

Larger ϵ:

- Wider tube
- More tolerance

Smaller ϵ:

- Narrower tube
- More sensitive fit



Linear SVR with different ϵ values.

Kernel SVR

When the relationship is nonlinear, SVR can use kernels.

Prediction function:

$$f(x) = \sum_{i=1}^{m} (\alpha_i - \alpha_i^*) k(x_i, x) + b$$

Common kernels:

- RBF
- Polynomial

Kernel SVR is powerful for small to medium datasets but can become slow as the dataset grows.

Polynomial SVR

A polynomial kernel can model nonlinear regression behavior.

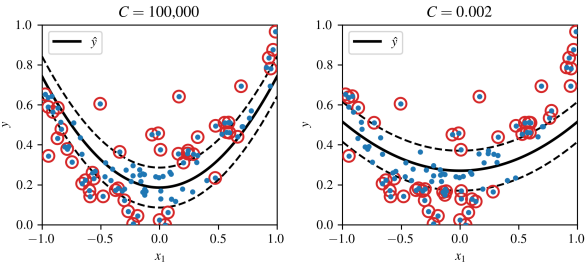
The parameter C controls regularization.

Larger C:

- Less regularization
- Tighter fit

Smaller C:

- More regularization
- Smoother fit



Polynomial-kernel SVR.

Example 7.4: SVM Polynomial Regression

- Fit noisy quadratic data using SVR
- Use a polynomial kernel
- Introduce an ϵ-insensitive margin
- Visualize the regression curve and tolerance region

This example demonstrates how SVR can model nonlinear relationships while controlling how closely the model follows the data.